Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

**Лабораторная работа №4**
**По дисциплине:** «Аппаратно-программное обеспечение ЭВМ и сетей»
**Тема:** «Архитектура и программирование сопроцессора. Использование целочисленных и вещественных команд»

**Выполнил:**
Студент 4 курса
Группы АС-66
Гончерёнок К.А.
**Проверила:**
Кулик А.Д.

**Брест 2026**

**Цель работы:** изучение архитектуры и программной модели сопроцессора; практическая работа с системой целочисленных и вещественных команд.

**Вариант 20 (2)**

**Постановка задачи. Раздел 1.**

**1.** Изучить следующие теоретические сведения: архитектура и программная модель сопроцессора – регистры и их назначение; форматы чисел, типовые команды, целочисленные команды (используя материалы лекций и методические указания).

**2.** Составить и отладить программу на языке ассемблера для вычисления значения функции, используя регистры сопроцессора и целочисленные команды согласно варианту. Оформить ее в виде отдельной функции. Каждую ветку алгоритма оформить в виде отдельной внутренней процедуры. Ввод и вывод данных осуществляется через консоль. Программу протестировать по всем условиям.

**Раздел 2. 1.** Изучить систему вещественных команд сопроцессора (используя материалы лекций и методические указания).

**2.** Составить и отладить программу на языке ассемблера для вычисления значения функции, используя регистры сопроцессора и вещественные команды согласно варианту. Оформить ее в виде отдельной функции. Каждую ветку алгоритма оформить в виде отдельной внутренней процедуры. Ввод и вывод данных осуществляется через консоль.

**Задание:**

**Вариант** 2

$$y = \begin{cases} 4xy + 5, & x + y > 10; \\ \dfrac{3xy + 2y}{x^2 + y^2 + 1}, & x + y < -2; \\ 6x - 2y^2 + 1, & -2 \le x + y \le 1. \end{cases}$$

**Код программы:**

```
; NASM, Linux x86-64. Standalone version: uses syscalls, no libc.
; Reads two decimal values x and y from stdin.

default rel

global _start

section .data
    prefix      db "Rezultat y = "
    prefix_len  equ $ - prefix
    outside_msg db "x+y vne oblasti opredeleniya (1 < x+y < 10).", 10
```

```
    outside_len equ $ - outside_msg
    input_msg   db "Ozhidalis dva chisla x i y.", 10
    input_len   equ $ - input_msg
    scale_int   dd 10000
    one         dq 1.0
    two         dq 2.0
    three       dq 3.0
    four        dq 4.0
    five        dq 5.0
    six         dq 6.0
    ten         dq 10.0
    minus_two   dq -2.0

section .bss
    input_buf   resb 256
    x_scaled    resq 1
    y_scaled    resq 1
    x_val       resq 1
    y_val       resq 1
    sum_val     resq 1
    numerator   resq 1
    denominator resq 1
    result      resq 1
    out_scaled  resq 1
    out_start   resq 1
    out_end     resq 1
    out_buf     resb 64

section .text
_start:
    ; Read a line (or as much input as fits).
    xor eax, eax                    ; sys_read
    xor edi, edi                    ; stdin
```

```
lea rsi, [input_buf]

mov edx, 255

syscall

test rax, rax

jle .bad_input

lea rbx, [input_buf]

add rbx, rax

mov byte [rbx], 0


lea rsi, [input_buf]

call parse_number

test rdx, rdx

jz .bad_input

mov [x_scaled], rax


call parse_number

test rdx, rdx

jz .bad_input

mov [y_scaled], rax


; Convert fixed-point input values to x87 floating point.

fild qword [x_scaled]

fidiv dword [scale_int]

fstp qword [x_val]

fild qword [y_scaled]

fidiv dword [scale_int]

fstp qword [y_val]


fld qword [x_val]

fadd qword [y_val]

fstp qword [sum_val]


; x+y > 10: y = 4xy + 5
```

```
    fld qword [sum_val]
    fcom qword [ten]
    fstsw ax
    sahf
    fstp st0
    ja .branch1

    ; x+y < -2: y = (3xy + 2y) / (x^2 + y^2 + 1)
    fld qword [sum_val]
    fcom qword [minus_two]
    fstsw ax
    sahf
    fstp st0
    jb .branch2

    ; -2 <= x+y <= 1: y = 6x - 2y^2 + 1
    fld qword [sum_val]
    fcom qword [one]
    fstsw ax
    sahf
    fstp st0
    jbe .branch3

    mov eax, 1                      ; sys_write
    mov edi, 1                      ; stdout
    lea rsi, [outside_msg]
    mov edx, outside_len
    syscall
    xor edi, edi
    jmp .exit

.branch1:
    fld qword [x_val]
```

```
    fmul qword [y_val]
    fmul qword [four]
    fadd qword [five]
    fstp qword [result]
    jmp .print_result

.branch2:
    ; Numerator: 3*x*y + 2*y
    fld qword [x_val]
    fmul qword [y_val]
    fmul qword [three]
    fld qword [y_val]
    fmul qword [two]
    faddp st1, st0
    fstp qword [numerator]

    ; Denominator: x*x + y*y + 1
    fld qword [x_val]
    fmul qword [x_val]
    fld qword [y_val]
    fmul qword [y_val]
    faddp st1, st0
    fadd qword [one]
    fstp qword [denominator]

    fld qword [numerator]
    fdiv qword [denominator]
    fstp qword [result]
    jmp .print_result

.branch3:
    fld qword [x_val]
    fmul qword [six]
```

```
    fld qword [y_val]
    fmul qword [y_val]
    fmul qword [two]
    fsubp st1, st0
    fadd qword [one]
    fstp qword [result]

.print_result:
    ; Convert to a four-decimal fixed-point integer for output.
    fld qword [result]
    fimul dword [scale_int]
    fistp qword [out_scaled]

    mov rax, [out_scaled]
    xor r8d, r8d
    test rax, rax
    jns .positive_result
    mov r8b, 1
    neg rax
.positive_result:
    xor edx, edx
    mov ecx, 10000
    div rcx
    mov r9, rax                    ; integer portion
    mov r10, rdx                   ; four-digit fraction

    lea rdi, [out_buf + 63]
    mov byte [rdi], 10
    dec rdi
    mov eax, r10d
    mov ecx, 10
    mov r11d, 4
.fraction_digits:
```

```
    xor edx, edx
    div ecx
    add dl, '0'
    mov [rdi], dl
    dec rdi
    dec r11d
    jnz .fraction_digits
    mov byte [rdi], '.'
    dec rdi

    mov rax, r9
.integer_digits:
    xor edx, edx
    mov ecx, 10
    div rcx
    add dl, '0'
    mov [rdi], dl
    dec rdi
    test rax, rax
    jnz .integer_digits
    test r8b, r8b
    jz .output
    mov byte [rdi], '-'
    dec rdi

.output:
    lea rsi, [rdi + 1]
    mov [out_start], rsi
    lea rax, [out_buf + 64]
    mov [out_end], rax

    mov eax, 1                      ; sys_write prefix
    mov edi, 1
```

```
    lea rsi, [prefix]
    mov edx, prefix_len
    syscall

    mov rsi, [out_start]
    mov rdx, [out_end]
    sub rdx, rsi
    mov eax, 1
    mov edi, 1
    syscall
    xor edi, edi
    jmp .exit

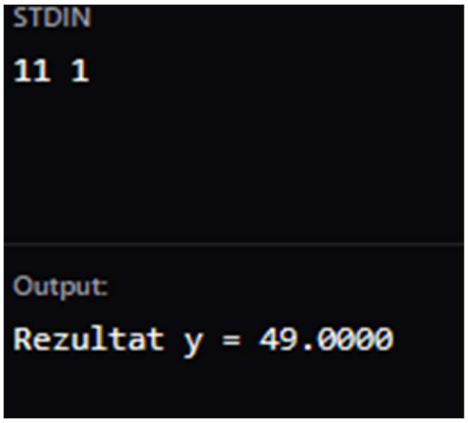
.bad_input:
    mov eax, 1                      ; sys_write
    mov edi, 1
    lea rsi, [input_msg]
    mov edx, input_len
    syscall
    mov edi, 1
.exit:
    mov eax, 60                     ; sys_exit
    syscall

; Parse one signed decimal number into RAX scaled by 10000.
; RDX is nonzero if at least one digit was read. RSI advances over the number.
parse_number:
.skip_space:
    movzx eax, byte [rsi]
    cmp al, ' '
    je .skip_one
    cmp al, 9
    jb .check_sign
```

```
    cmp al, 13
    jbe .skip_one
    jmp .check_sign
.skip_one:
    inc rsi
    jmp .skip_space
.check_sign:
    xor r12d, r12d
    cmp al, '-'
    jne .check_plus
    mov r12d, 1
    inc rsi
    jmp .start_digits
.check_plus:
    cmp al, '+'
    jne .start_digits
    inc rsi
.start_digits:
    xor r8d, r8d                    ; integer magnitude
    xor r9d, r9d                    ; fraction magnitude
    xor r10d, r10d                  ; fraction digit count (max 4)
    xor r11d, r11d                  ; saw digit flag
.integer_loop:
    movzx eax, byte [rsi]
    cmp al, '0'
    jb .decimal_point
    cmp al, '9'
    ja .decimal_point
    imul r8, r8, 10
    sub eax, '0'
    add r8, rax
    inc rsi
    mov r11d, 1
```

```
    jmp .integer_loop
.decimal_point:
    cmp al, '.'
    jne .finish_fraction
    inc rsi
.fraction_loop:
    movzx eax, byte [rsi]
    cmp al, '0'
    jb .finish_fraction
    cmp al, '9'
    ja .finish_fraction
    cmp r10d, 4
    jae .fraction_consume
    imul r9, r9, 10
    sub eax, '0'
    add r9, rax
    inc r10d
.fraction_consume:
    inc rsi
    mov r11d, 1
    jmp .fraction_loop
.finish_fraction:
.pad_fraction:
    cmp r10d, 4
    jae .make_result
    imul r9, r9, 10
    inc r10d
    jmp .pad_fraction
.make_result:
    mov rax, r8
    imul rax, rax, 10000
    add rax, r9
    test r12d, r12d
```

```
    jz .set_valid
    neg rax
.set_valid:
    mov rdx, r11
    ret
```



**Вывод:** в ходе лабораторной работы изучены архитектура и программная модель математического сопроцессора x87: стек из восьми 80-битных регистров ST(0)–ST(7), порядок загрузки и выгрузки данных, вычисления через вершину стека, сравнение чисел командой fcomip с передачей результата в флаги EFLAGS и условными переходами. Практически освоены две системы команд. Целочисленные (fild, fiadd, fimul, fistp) берут операнды-целые из памяти и сами переводят их в вещественный формат, но не учитывают дробную часть данных. Вещественные (fld, fadd, fmul, fdiv, fchs) работают с форматом double, и от дробной части зависят и значение функции, и выбор ветки.